

CS 3563: Introduction to DBMS II

Assignment 2: Query Optimization Analysis

Due Date and Time: 16 March 2026, 11:59 PM IST

Instructions

Total Marks: 60

1. The assignment must be completed in the assigned groups.
2. All experiments must be performed on PostgreSQL version 14 or higher.
3. The submission must be a ZIP file containing all required files, organized according to the folder structure specified in the Submission Guidelines section, and submitted via Moodle.
4. Late submissions will be handled in accordance with the course policy.

Background

For a given SQL query, the database query optimizer aims to identify an optimal query execution plan. It does so by systematically evaluating various query plans for their execution cost and selecting the minimum cost plan. However, it often makes errors in cost estimation, which leads to the selection of a suboptimal plan for the query.

In this assignment, you will inspect different query plans chosen by the optimizer and analyze their optimality.

Definitions and Notations.

Parameterized query, query instance and query plan. Let q represent a parameterized query, and q_i represents an instance of q with parameter i . Further, let P_i denote the optimizer's chosen plan for query instance q_i .

Neighboring query instances. Two query instances q_i and q_j are considered neighbours if their parameter values differ by a unit step. That is, there exists no valid parameter value in the domain of the parameterized attribute between i and j . If a query has multiple variable parameters, query instances are considered neighbours if they differ only in one parameter, differing by a unit step. We denote neighborhood as a function $N(q_i, q_j)$ which returns T if q_i and q_j are neighbours, F otherwise.

Plan Equivalence. Two query plans P_i and P_j are considered equivalent if they are structurally identical, i.e. the operator algorithms and their sequence are identical, and they differ in their parameter values (i and j) only. If two query plans are equivalent, we denote them as $P_i \equiv P_j$.

Plan Switches. For a parameterized range query q , as you vary the parameter value, the optimizer switches between different execution plans. We denote a plan switch as $S_{i,j}$. This means that query instances q_i and q_j are such that $P_i \neq P_j$ and q_i, q_j are a pair of neighbouring instances of query q , i.e. $N(q_i, q_j) = T$.

Plan Cost Function. For a query plan P , the compile-time estimated cost of executing that plan for a query instance q_i is given as $Cost(P, q_i)$.

A Plan Cost Function exhibits monotonicity when the estimated cost of each plan varies monotonically with the selectivity of the variable predicate. That is, for a pair of query instances q_i and q_j , with selectivities of the variable predicate s_i and s_j respectively, if $s_j \geq s_i$, then $Cost(P, s_j) \geq Cost(P, s_i)$ for all query plans P .

Plan Running-Time Function. The actual running-time (wall-clock time) of executing a plan P for a query instance q_i is denoted as $RT(P, q_i)$.

Similar to Plan Cost Function, the Running Time function exhibits monotonicity, when the following condition holds: For a pair of query instances q_i and q_j , with selectivities of the variable predicate s_i and s_j respectively, if $s_j \geq s_i$, then $RT(P, s_j) \geq RT(P, s_i)$ for all query plans P .

Task 0: Database Setup and Configuration

0 Marks

Objective

Install PostgreSQL database management system, configure the `pg_hint_plan` extension for forcing specific query execution plans, and load the TPC-H benchmark tables.

PostgreSQL and `pg_hint_plan` Setup

- Install PostgreSQL version 14 or higher on your system.
- Install and configure the `pg_hint_plan` extension. The `pg_hint_plan` is a PostgreSQL extension that allows you to force specific operator algorithms or join orders that were not natively chosen by the query optimizer. This forcing is achieved by specifying hints in SQL comments.
- Follow the PostgreSQL and `pg_hint_plan` setup guide: <https://drive.google.com/file/d/1BquaZiC64dSBIJgpk5btseIDIq2-gJkk/view?usp=sharing>

TPC-H Database Setup

- Follow the TPC-H setup guide to create and load the required tables (customer, orders, lineitem): <https://drive.google.com/file/d/1j2e6KfBLB1P4B1bVwj6QpnwdirXbUYAe/view?usp=sharing>
- Create the indexes as mentioned in the TPC-H setup guide. These indexes are required for the experiments in this assignment.

Assignment Queries

In this assignment, you will be working with three (modified) TPC-H parameterized queries, available at the following link:

Assignment Queries Document:

<https://docs.google.com/document/d/1rNkWvDV8Ei3NBizvnnAwQ9i66jbMHf6kxU001UK06CY/edit?usp=sharing>

Task 1: Plan Comparison

(15 Marks)

Objective

Given two PostgreSQL compile-time¹ query plans P_i and P_j , implement a function to determine if $P_i \equiv P_j$, i.e. if they are structurally identical.

Instructions

1. Implement a function that takes a PostgreSQL compile-time plan for a query instance, as input, and computes its physical operator tree, for plan comparison subsequently. (Here, the physical operator tree refers to its structure that discounts the query parameter values and other values such as the cost and cardinality estimates that are available in the query plans produced by PostgreSQL.)
2. Next, implement a comparison function that takes two compile-time query plans from Postgres as input, and correctly identifies plan equivalence. (This function will internally call the function from (1).)
3. A starter code file with the required function signature is provided: <https://drive.google.com/file/d/1sx3mc2zNLtd08SsTdzzqDbz8T0iMwXT0X/view?usp=sharing>
4. Your program must read query plans from two JSON files: `query1.json` and `query2.json`.
5. The program should output exactly one of the following:
 - YES: Both plans are structurally identical
 - NO: Plans are structurally different
6. Validate your implementation with appropriate test cases.

Deliverables

- Source code file (e.g., `task1.py`, `task1.java`, `task1.cpp`)
- README.md explaining your approach and how to run the program
- Sample test case files:
 - `plan1.json` - First query execution plan
 - `plan2.json` - Second query execution plan

¹ Compile-time plans are execution plans chosen by the optimizer during planning phase. These are produced before actually running the query. These also contain the optimizer's estimates with respect to various operator costs and output row cardinality. Refer to the PostgreSQL and `pg_hint_plan` guide for details on obtaining compile-time execution plans.

Task 2: Plan Switch Detection

(15 Marks)

Objective

For a given parameterized range query, identify all switches where the PostgreSQL query optimizer changes its execution plan strategy as the query parameter varies.

Assumption

You may assume that plan cost monotonicity holds.

Instructions

1. Determine the valid parameter range for the query by examining the dataset.
2. Implement an efficient algorithm to explore the parameter space to detect plan switches. (The Plan Comparator code can be used to compare plans here.)
3. Perform this computation for all three assignment queries.

Output Format

The `output.txt` file should list all detected plan switches for each query in the following format:

Query 1:

Plan Switch 1: YYYY-MM-DD and YYYY-MM-DD

Plan Switch 2: YYYY-MM-DD and YYYY-MM-DD

...

Query 2:

Plan Switch 1: YYYY-MM-DD and YYYY-MM-DD

Plan Switch 2: YYYY-MM-DD and YYYY-MM-DD

...

Query 3:

Plan Switch 1: YYYY-MM-DD and YYYY-MM-DD

Plan Switch 2: YYYY-MM-DD and YYYY-MM-DD

...

Implementation Note. You may choose to implement this task using:

- A single source code file that analyzes all three queries, OR
- Three separate source code files, one for each query

Both approaches are acceptable. If using separate files, name them appropriately (e.g., `task2_query1.py`, `task2_query2.py`, `task2_query3.py`).

Deliverables

- Source code file(s):
 - Single file approach: `task2.py` (or `.java`, `.cpp`, etc.)
 - Multiple file approach: `task2_query1.py`, `task2_query2.py`, `task2_query3.py`
- `output.txt` listing all detected plan switches for each query
- `analysis.pdf` containing:
 - For each plan switch: the switch dates, the plan chosen before the switch, and the plan chosen after the switch.
- `README.md` documenting your methodology and implementation approach.

Task 3: Plan Switch Classification

(15 Marks)

Objective

For each plan switch identified in Task 2, analyze whether the optimizer's decision to switch plans was made at the right time.

Notation Let $S_{i,j}$ denote a plan switch identified in Task 2. This means that (q_i, q_j) are such that $N(q_1, q_j) = T$ and $P_i \neq P_j$, where P_i and P_j denote the plans chosen by the optimizer at q_1 and q_2 respectively.

Instructions

1. For each plan switch $S_{i,j}$ identified in Task 2, measure the following:
 - (a) $RT(P_i, q_i)$: Running time for executing P_i at q_i .
 - (b) $RT(P_j, q_j)$: Running time for executing P_j at q_j .
 - (c) $RT(P_i, q_j)$: Running time for executing P_i at q_j .
 - (d) $RT(P_j, q_i)$: Running time for executing P_j at q_i .Note that while the first two can be obtained by directly executing query instances q_i and q_j , the latter two would require the plan forcing using `pg_hint_plan`.
2. Analyze the four execution times to determine whether the optimizer made the correct switching decision.
3. Classify each plan switches across all three queries, into one of the following classes:
 - **DELAYED**: The optimizer made the plan switch late. The alternative plan P_j was already outperforming the optimizer's chosen plan P_i before the plan switch, meaning the plan switch should have occurred earlier.
 - **EARLY**: The optimizer made the plan switch early. The original plan P_i was still outperforming the alternative plan P_j , at the plan switch, meaning the plan switch should have occurred later.
 - **CORRECT**: The optimizer made the plan switch at the right time. On each side of the plan switch, the optimizer's chosen plan was indeed the better-performing plan.

Deliverables

- **analysis.pdf** containing:
 - A table for each assignment query: plan switch $S_{i,j}$, the four execution times $RT(P_i, q_i)$, $RT(P_j, q_j)$, $RT(P_i, q_j)$, $RT(P_j, q_i)$.
 - Classification of each plan switch as **DELAYED**, **EARLY**, or **CORRECT**, with justification based on the measured execution times.
 - **pg_hint_plan** directives used for forcing each plan.
- **queries.sql** containing all executed queries with hints for each plan switch.

Task 4: Plan Switch Correction

(15 Marks)

Objective

For each of the incorrect plan switches identified in Task 3, determine the corresponding optimal plan switch.

Assumption

You may assume that the plan running-time function exhibits monotonicity.

Instructions

1. For each incorrectly timed switch $S_{i,j}$, determine the **optimal plan switch**
 - the parameter values $q_{i'}$, $q_{j'}$ at which the plan switch should have occurred based on actual performance.
2. Measure the execution times $RT(P_{i'}, q_{i'})$, $RT(P_{j'}, q_{j'})$, $RT(P_{i'}, q_{j'})$, and $RT(P_{j'}, q_{i'})$ at the optimal switch point to validate the correctness of the identified optimal plan switch.

Implementation Note. You may implement this task using a single generalized code file, separate files per query, or any other modular organization. All approaches are acceptable as long as the analysis is complete and correct.

Deliverables

1. Source code file(s) (e.g., `task4.py`, or `task4_query1.py`, `task4_query2.py`, `task4_query3.py`, etc.)
2. **analysis.pdf** containing for each query:
 - For each incorrect switch: the optimizer's plan switch and your identified optimal switch.
 - Execution times at the optimal switch.

Submission Guidelines

Folder Structure

Your submission must follow this exact structure:

```
<member1RollNo>-<member2RollNo>/
|
+-- task1/
|   +-- task1.py (or .java, .cpp, etc.)
|   +-- plan1.json
|   +-- plan2.json
|   +-- README.md
|
+-- task2/
|   +-- task2.py (or .java, .cpp, etc.)
|   |   OR
|   +-- task2_query1.py, task2_query3.py, task2_query4.py
|   +-- analysis.pdf
|   +-- output.txt
|   +-- README.md
|
+-- task3/
|   +-- analysis.pdf
|   +-- queries.sql
|
+-- task4/
|   +-- task4.py (or task4_query1.py, task4_query2.py, task4_query3.py, etc.)
|   +-- analysis.pdf
```

Submission Process

1. Verify your folder structure matches the specification exactly
2. Create a ZIP file of your entire submission folder
3. Name the ZIP file as: <member1RollNo>-<member2RollNo>.zip
4. Submit only the ZIP file via Moodle before the deadline
5. **Important:** Only one member of the group should submit the assignment. If both members submit, one submission will be chosen at random for evaluation.